TP 0 : Mise en Place

Création & Compréhension du projet ShopFlow

Objectif du TP

À la fin de ce TP, vous aurez créé de A à Z le projet ShopFlow, compris son architecture en couches (routes, services, modèles), et serez capable d'expliquer le rôle de chaque fichier. Ce projet sera le fil rouge de tous les TPs suivants (TP1 à TP4).

Stack : Python 3.11 · FastAPI · SQLAlchemy · SQLite · Pydantic v2 · Redis

Prérequis : Python installé · Git configuré · VS Code ou PyCharm

Présentation de ShopFlow

ShopFlow est une API e-commerce simplifiée que vous allez construire pas à pas. Elle gère trois grandes fonctionnalités : un **catalogue de produits**, un **panier** par utilisateur, et un système de **commandes**. Un système de coupons de réduction sera ajouté en TP4 via le cycle TDD.

Ce que fait ShopFlow

- GET /products/ : liste tous les produits du catalogue (avec filtre catégorie, prix, pagination)
- POST /products/ : ajouter un produit au catalogue (admin)
- POST /cart/?user_id=X : ajouter un produit au panier d'un utilisateur
- GET /cart/{user_id} : afficher le contenu du panier avec le total TTC
- POST /orders/ : convertir le panier en commande (vérifie le stock, calcule le total, vide le panier)
- PATCH /orders/{id}/status : faire évoluer le statut : pending → confirmed → shipped
- POST /coupons/apply : appliquer un coupon de réduction sur un prix
- GET /health : smoke test, vérifie que l'API est opérationnelle

Architecture en couches

ShopFlow suit une architecture en **3 couches** : cette séparation est essentielle pour la testabilité (TP1 à TP4).

Couche	Fichiers	Rôle

Routes (API)	`app/routes/products.py`
	`app/routes/cart.py`
	`app/routes/orders.py`
		Reçoit les requêtes HTTP, valide via Pydantic, appelle les services, retourne la réponse

Services (Métier)	`app/services/pricing.py`
	`app/services/stock.py`
	`app/services/cart.py`
		Contient toute la logique métier, testée unitairement dans TP1

Modèles (BDD)	`app/models.py` `app/database.py`	Définit les tables SQLite via SQLAlchemy ORM
Schémas (Contrat)	`app/schemas.py`	Pydantic : valide les requêtes entrantes et formate les réponses
Cache	`app/cache.py`	Client Redis pour accélérer les lectures, mocké dans TP1

Partie 1 : Initialisation du Projet

1.1 Créer la structure de dossiers

Depuis votre terminal, créez la structure complète du projet :

```
Terminal
# Créer le dossier racine
mkdir shopflow && cd shopflow

# Créer l'arborescence complète en une seule commande
mkdir -p app/routes app/services tests/unit tests/integration tests/perf

# Créer tous les fichiers __init__.py (packages Python)
touch app/__init__.py app/routes/__init__.py app/services/__init__.py
touch tests/__init__.py tests/unit/__init__.py
touch tests/integration/__init__.py tests/perf/__init__.py

# Créer les fichiers principaux vides
touch app/database.py app/models.py app/schemas.py app/cache.py app/main.py
touch app/services/pricing.py app/services/stock.py
touch app/services/cart.py app/services/order.py
touch app/routes/products.py app/routes/cart.py
touch app/routes/orders.py app/routes/coupons.py
touch pytest.ini .coveragerc .gitignore .env.example
touch Dockerfile docker-compose.yml Makefile requirements.txt README.md

# Vérifier la structure créée
find . -not -path './.git/*' | sort
```

1.2 Environnement virtuel et dépendances

Créez et activez le venv, puis créez le fichier `requirements.txt` :

```
Terminal
# Créer et activer le venv
python -m venv venv
source venv/bin/activate          # Linux / Mac
venv\Scripts\activate            # Windows

# Vérifier que le bon Python est utilisé
which python                      # doit pointer vers venv/bin/python
```

Copiez le contenu suivant dans `requirements.txt` :

```
requirements.txt
fastapi==0.110.0
uvicorn[standard]==0.29.0
sqlalchemy==2.0.28
pydantic==2.6.4
```

```
pydantic-settings==2.2.1
redis==5.0.3
httpx==0.27.0
faker==24.3.0
pytest==8.1.1
pytest-cov==5.0.0
pytest-mock==3.12.0
pytest-asyncio==0.23.6
pytest-xdist==3.5.0
locust==2.24.1
bandit==1.7.8
pylint==3.1.0
flake8==7.0.0
```

Terminal — vérification

```
# Installer toutes les dépendances
pip install -r requirements.txt

# Vérifier les installations clés
python -c "import fastapi; print('FastAPI', fastapi.__version__)"
python -c "import sqlalchemy; print('SQLAlchemy', sqlalchemy.__version__)"
pytest --version
```

Pourquoi ces dépendances ?

- fastapi + uvicorn : framework web asynchrone + serveur ASGI haute performance
- sqlalchemy : ORM Python, traduit les classes Python en tables SQL
- pydantic : validation des données d'entrée/sortie de l'API (types stricts)
- redis : cache pour les lectures fréquentes (liste de produits, stock)
- pytest + pytest-cov + pytest-mock : suite de tests complète (TP1 à TP4)
- faker : génération de données de test réalistes (noms, prix, emails)
- locust : tests de charge (TP4), simuler des centaines d'utilisateurs
- bandit : analyse de sécurité statique du code Python (TP3 pipeline)

1.3 Fichiers de configuration

Créez le fichier `pytest.ini` à la racine du projet (déjà crée à la question 1.1, copier et coller les configurations) :

pytest.ini

```
[pytest]
addopts = -v --cov=app --cov-report=html --cov-report=term-missing --cov-
fail-under=80
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*

markers =
    unit: tests unitaires isolés
    integration: tests d'intégration API
    slow: tests lents
```

Créez le fichier `.coveragerc` :

```
.coveragerc
[run]
source = app
omit = */migrations/*, */__init__.py, */conftest.py

[report]
fail_under = 80
show_missing = true

[html]
directory = htmlcov
title = ShopFlow — Coverage Report
```

Relecture 1 sur Initialisation	**Commentaires**
1.1. Expliquez en 2 lignes la différence entre une dépendance de production (fastapi) et une dépendance de test (pytest). Pourquoi les séparer en pratique ?	Production = fait tourner l'app en prod (fastapi, sqlalchemy). Test = valide le code en dev (pytest). On les sépare pour ne pas alourdir l'image Docker et réduire les vulnérabilités.
1.2. À quoi sert le fichier __init__.py dans chaque dossier Python ? Que se passerait-il si on l'oubliait dans app/services/ ?	Déclare le dossier comme package Python importable. Sans lui dans app/services/ → ModuleNotFoundError à l'import et pytest ne trouve plus les tests.
1.3. Pourquoi utilise-t-on un environnement virtuel (venv) plutôt que d'installer les packages directement sur le système Python global ?	Isole les packages par projet pour éviter les conflits de versions entre projets. Bonus : reproductible sur toute machine via pip install -r requirements.txt, sans droits admin.

Partie 2 : Base de Données et Modèles

Dans cette partie, vous allez remplir les deux fichiers fondamentaux de ShopFlow : `app/database.py` qui configure la connexion SQLite, et `app/models.py` qui définit les tables via SQLAlchemy ORM.

2.1 Schéma relationnel de ShopFlow

Avant d'écrire le code, comprenez le schéma de la base de données :

Product	Cart	CartItem	Order	Coupon
id (PK)	id (PK)	id (PK)	id (PK)	code (PK)
name	user_id	cart_id (FK)	user_id	reduction
price	created_at	product_id (FK)	total_ht	actif
stock		quantity	total_ttc	created_at
category			coupon_code (FK)	
active			status	

Relations entre les tables

- Un Cart appartient à un user_id (utilisateur), 1 panier par utilisateur
- Un CartItem relie un Cart à un Product avec une quantité (table de liaison)
- Un Order est créé depuis un Cart, il contient les OrderItems (snapshot du panier)
- Un Coupon peut être appliqué à une Order via coupon_code (FK nullable)
- Les ForeignKey avec ondelete='CASCADE' : si on supprime un Cart, ses CartItems sont supprimés automatiquement

2.2 Remplir app/database.py

Copiez le code suivant dans `app/database.py`. Lisez chaque commentaire attentivement avant de continuer.

app/database.py

```
# app/database.py
import os
from sqlalchemy import create_engine, event
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from sqlalchemy.pool import StaticPool

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./shopflow.db")
# ↑ En développement : fichier shopflow.db
# ↑ En test (conftest.py) : sqlite:///:memory: (RAM isolée)
# ↑ En prod : peut être remplacé par postgresql://...

def create_db_engine(url=DATABASE_URL):
    if url == "sqlite:///:memory:":
        # StaticPool : connexion unique partagée (obligatoire pour :memory:)
        return create_engine(url,
            connect_args={"check_same_thread": False},
            poolclass=StaticPool)
    elif url.startswith("sqlite"):
        engine = create_engine(url,
            connect_args={"check_same_thread": False})
        @event.listens_for(engine, "connect")
        def set_fk(conn, _):
```

```
            conn.cursor().execute("PRAGMA foreign_keys=ON")
        return engine
    return create_engine(url)

engine       = create_db_engine()
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass  # Classe parente de tous les modèles SQLAlchemy

def get_db():
    """Générateur de session — utilisé via Depends(get_db) dans les
routes."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

2.3 Remplir app/models.py

Copiez le modèle `Product`, `Coupon`, `Cart`, `CartItem` et `Order` ci-dessous.

app/models.py : (tout est fournis)

```
# app/models.py
from datetime import datetime
from sqlalchemy import Column, Integer, String, Float, Boolean,
                       ForeignKey, DateTime, Text
from sqlalchemy.orm import relationship
from app.database import Base

class Product(Base):
    __tablename__ = "products"
    id          = Column(Integer, primary_key=True, index=True)
    name        = Column(String(100), nullable=False)
    description = Column(Text, nullable=True)
    price       = Column(Float, nullable=False)     # Prix HT
    stock       = Column(Integer, default=0)
    category    = Column(String(50), nullable=True)
    active      = Column(Boolean, default=True)
    created_at  = Column(DateTime, default=datetime.utcnow)
    cart_items  = relationship("CartItem", back_populates="product")
    order_items = relationship("OrderItem", back_populates="product")

class Coupon(Base):
    __tablename__ = "coupons"
    code       = Column(String(20), primary_key=True)
    reduction  = Column(Float, nullable=False)  # 0 < reduction <= 100
    actif      = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)



class Cart(Base):
    __tablename__ = 'carts'
```

```
    id         = Column(Integer, primary_key=True, index=True)
    user_id    = Column(Integer, nullable=False, index=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)
    items      = relationship('CartItem', back_populates='cart',
                              cascade='all, delete-orphan')

class CartItem(Base):
    __tablename__ = 'cart_items'
    id         = Column(Integer, primary_key=True, index=True)
    cart_id    = Column(Integer, ForeignKey('carts.id', ondelete='CASCADE'),
                        nullable=False)
    product_id = Column(Integer, ForeignKey('products.id'), nullable=False)
    quantity   = Column(Integer, nullable=False, default=1)
    cart       = relationship('Cart',    back_populates='items')
    product    = relationship('Product', back_populates='cart_items')

class Order(Base):
    __tablename__ = 'orders'
    id          = Column(Integer, primary_key=True, index=True)
    user_id     = Column(Integer, nullable=False, index=True)
    total_ht    = Column(Float, nullable=False)
    total_ttc   = Column(Float, nullable=False)
    coupon_code = Column(String(20), ForeignKey('coupons.code'),
nullable=True)
    status      = Column(String(20), default='pending')
    created_at  = Column(DateTime, default=datetime.utcnow)
    items       = relationship('OrderItem', back_populates='order',
                               cascade='all, delete-orphan')
    coupon      = relationship('Coupon')

class OrderItem(Base):
    __tablename__ = 'order_items'
    id         = Column(Integer, primary_key=True, index=True)
    order_id   = Column(Integer, ForeignKey('orders.id', ondelete='CASCADE'),
                        nullable=False)
    product_id = Column(Integer, ForeignKey('products.id'), nullable=False)
    quantity   = Column(Integer, nullable=False)
    unit_price = Column(Float, nullable=False)
    order      = relationship('Order',   back_populates='items')
    product    = relationship('Product', back_populates='order_items')
```

2.4 Vérifier les modèles

```
Terminal — vérification
# Tester que SQLAlchemy peut créer les tables sans erreur
python -c "
from app.database import engine, Base
from app.models import Product, Cart, CartItem, Order, OrderItem, Coupon
Base.metadata.create_all(engine)
print('Toutes les tables créées avec succès')
"
```

Relecture 2 : Base de données et modèles	**Commentaires**
2.1. Expliquez la différence entre Column(ForeignKey('carts.id')) et Column(ForeignKey('carts.id', ondelete='CASCADE')). Dans quel cas le CASCADE est-il important pour ShopFlow ?	Sans CASCADE : supprimer un Cart bloque sur les CartItems existants, c'est une erreur d'intégrité. Avec CASCADE : supprimer un Cart supprime automatiquement ses CartItems. Indispensable dans ShopFlow quand on vide un panier.
2.1. Pourquoi OrderItem possède-t-il un champ unit_price alors que Product possède déjà un champ price ? Quel problème réel cela évite-t-il ?	product.price peut changer après la commande. unit_price capture le prix au moment de l'achat. Sans ça, une facture afficherait le prix actuel au lieu du prix payé, erreur comptable et légale.
2.2. Dans database.py, à quoi sert le PRAGMA foreign_keys=ON ? Que se passerait-il si on l'omettait lors d'une suppression en cascade ?	PRAGMA foreign_keys=ON SQLite désactive les contraintes FK par défaut. Sans ce PRAGMA, on peut insérer un CartItem avec un cart_id inexistant sans erreur, et le CASCADE ne fonctionnerait pas à la suppression.

Partie 3 : Schémas Pydantic

Les schémas Pydantic définissent le **contrat de l'API** : ce qu'elle accepte en entrée et ce qu'elle retourne en sortie. Ils sont distincts des modèles SQLAlchemy.

3.1 Modèle SQLAlchemy vs Schéma Pydantic

Modèle SQLAlchemy (models.py)	Schéma Pydantic (schemas.py)
Représente une TABLE en base	Représente un CONTRAT HTTP (JSON)
Héritage de Base (SQLAlchemy)	Héritage de BaseModel (Pydantic)
Colonnes = Column(Integer, ...)	Champs = prix: float = Field(gt=0)
Pas de validation automatique	Validation stricte des types à l'entrée
Persisté en BDD	Jamais stocké, uniquement en mémoire
Relations via relationship()	Relations via List[AutreSchema]

3.2 Remplir app/schemas.py

Les schémas Product Cart et Order sont fournis ci-dessous.

app/schemas.py (tout fournis)

```python
# app/schemas.py
from datetime import datetime
from typing import Optional, List
from pydantic import BaseModel, Field, field_validator

# PRODUCT
class ProductCreate(BaseModel):
    name:     str   = Field(..., min_length=1, max_length=100)
    price:    float = Field(..., gt=0, description="Prix HT")
    stock:    int   = Field(default=0, ge=0)
    category: Optional[str] = None
    active:   bool  = True

class ProductResponse(ProductCreate):
    id: int
    created_at: datetime
    model_config = {"from_attributes": True}
    # ↑ Permet de convertir un objet SQLAlchemy directement en schema

# COUPON
class CouponCreate(BaseModel):
    code:      str   = Field(..., min_length=1, max_length=20)
    reduction: float = Field(..., gt=0, le=100)
    actif:     bool  = True

    @field_validator("code")
    @classmethod
    def code_uppercase(cls, v: str) -> str:
        return v.upper()  # PROMO20, pas promo20
```

```python
class CouponResponse(BaseModel):
    code: str
    reduction: float
    actif: bool
    model_config = {"from_attributes": True}

class CartItemCreate(BaseModel):
    product_id: int = Field(..., gt=0)
    quantity:   int = Field(..., gt=0, le=100)

class CartItemResponse(BaseModel):
    id:         int
    product_id: int
    quantity:   int
    product:    Optional[ProductResponse] = None
    model_config = {'from_attributes': True}

class CartResponse(BaseModel):
    id:         int
    user_id:    int
    items:      List[CartItemResponse] = []
    sous_total: float = 0.0
    model_config = {'from_attributes': True}

# ORDER

class OrderCreate(BaseModel):
    user_id:     int           = Field(..., gt=0)
    coupon_code: Optional[str] = Field(None, max_length=20)

class OrderItemResponse(BaseModel):
    id:         int
    product_id: int
    quantity:   int
    unit_price: float
    model_config = {'from_attributes': True}

class OrderResponse(BaseModel):
    id:          int
    user_id:     int
    total_ht:    float
    total_ttc:   float
    coupon_code: Optional[str]
    status:      str
    created_at:  datetime
    items:       List[OrderItemResponse] = []
    model_config = {'from_attributes': True}
```

3.3 Vérifier les schémas

```
Terminal — vérification des schémas
# Tester la validation Pydantic
python -c "
```

```
from app.schemas import ProductCreate, CouponCreate

# Valide
p = ProductCreate(name='Laptop', price=999.99, stock=10)
print('Produit valide:', p)

# Coupon avec code en minuscules → automatiquement en majuscules
c = CouponCreate(code='promo20', reduction=20.0)
print('Code coupon:', c.code)  # doit afficher PROMO20

# Test de validation : prix négatif doit échouer
try:
    ProductCreate(name='X', price=-5.0, stock=1)
except Exception as e:
    print('Validation OK — prix négatif rejeté:', type(e).__name__)
"
```

Relecture 3 : Schémas Pydantic	**Commentaires**
3.1. Expliquez ce que fait model_config = {"from_attributes": True} dans ProductResponse. Que se passerait-il sans cette ligne quand FastAPI retourne un objet SQLAlchemy ?	from_attributes Permet à Pydantic de lire les attributs d'un objet SQLAlchemy (product.name) au lieu d'un dictionnaire. Sans cette ligne, ValidationError car Pydantic ne sait pas lire un objet ORM.
3.2. Dans CartItemCreate, on utilise Field(..., gt=0, le=100) pour quantity. Expliquez ce que signifient gt=0 et le=100 et quel comportement cela déclenche si on envoie quantity=0 ou quantity=200.	gt=0 : quantity doit être strictement supérieur à 0 le=100 : quantity doit être inférieur ou égal à 100 Si on envoie quantity=0 ou quantity=200, FastAPI retourne automatiquement 422 Unprocessable Entity avec le message d'erreur précis, sans écrire une seule ligne de validation.
3.3. Expliquez pourquoi on a deux classes distinctes ProductCreate et ProductResponse au lieu d'une seule. Quel champ est présent dans Response mais pas dans Create et pourquoi ?	ProductCreate : contient ce que l'utilisateur envoie (name, price, stock…) ProductResponse : contient en plus id et created_at, générés côté serveur par la BDD Si on n'avait qu'une seule classe, le client pourrait envoyer son propre id ou falsifier created_at, risque de sécurité et de collision en base.

Partie 4 : Services Métier

Les services contiennent la **logique métier pure** de ShopFlow. Ils ne connaissent pas HTTP, ils reçoivent des objets Python et retournent des objets Python. C'est pour cette raison qu'ils sont facilement testables unitairement (TP1).

4.1 Remplir app/services/pricing.py

Le module pricing calcule les prix. Copiez le code fourni :

app/services/pricing.py — fonctions fournies

```
# app/services/pricing.py
from typing import Optional, List, Tuple
from app.models import Product, Coupon

TVA_RATE = 0.20   # Taux TVA France (20%)

def calcul_prix_ttc(prix_ht: float) -> float:
    """Calcule le prix TTC à partir du prix HT. Lève ValueError si prix <
0."""
    if prix_ht < 0:
        raise ValueError(f"Prix HT invalide : {prix_ht}")
    return round(prix_ht * (1 + TVA_RATE), 2)

def appliquer_coupon(prix: float, coupon: Coupon) -> float:
    """Applique une réduction. Lève ValueError si coupon inactif ou réduction
invalide."""
    if not coupon.actif:
        raise ValueError(f"Coupon inactif : {coupon.code}")
    if not 0 < coupon.reduction <= 100:
        raise ValueError(f"Réduction invalide : {coupon.reduction}")
    return round(prix * (1 - coupon.reduction / 100), 2)

def calculer_total(
    produits: List[Tuple[Product, int]],
    coupon: Optional[Coupon] = None
) -> float:
    if not produits:
        return 0.0                              # liste vide → 0
    total_ht = sum(p.price * q for p, q in produits)  # somme HT
    total_ttc = calcul_prix_ttc(total_ht)      # → TTC
    if coupon:
        total_ttc = appliquer_coupon(total_ttc, coupon)  # réduction
    return total_ttc
```

4.2 Remplir app/services/stock.py

Copiez le code :

app/services/stock.py — verifier_stock et liberer_stock (fournis)

```
# app/services/stock.py
import logging
```

```
from sqlalchemy.orm import Session
from app.models import Product
from app.cache import redis_client

logger = logging.getLogger(__name__)

def verifier_stock(product: Product, quantite: int) -> bool:
    """Retourne True si stock >= quantite, False sinon."""
    if quantite <= 0:
        raise ValueError(f"Quantité invalide : {quantite}")
    return product.stock >= quantite

def liberer_stock(product: Product, quantite: int, session: Session) ->
Product:
    """Libère du stock (annulation) et met à jour le cache Redis."""
    if quantite <= 0:
        raise ValueError(f"Quantité invalide : {quantite}")
    product.stock += quantite
    session.commit()
    session.refresh(product)
    redis_client.set(f"product:{product.id}:stock", product.stock)
    return product

def reserver_stock(product: Product, quantite: int, session: Session) ->
Product:
    if not verifier_stock(product, quantite):
        raise ValueError(
            f"Stock insuffisant pour '{product.name}' : "
            f"{product.stock} disponible(s), {quantite} demandé(s)."
        )
    product.stock -= quantite
    session.commit()
    session.refresh(product)
    cache_key = f"product:{product.id}:stock"
    redis_client.delete(cache_key)   # invalider le cache
    logger.info(f"Stock réservé : {product.id}, qty={quantite}")
    return product
```

4.3 Remplir app/cache.py

Copiez ce fichier tel quel. Il sera **mocké** dans les tests (TP1), vous n'avez pas besoin que Redis soit démarré pour développer.

```
app/cache.py
# app/cache.py
import os, logging
from unittest.mock import MagicMock

logger = logging.getLogger(__name__)
REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379")

def _create_redis_client():
    try:
```

```
        import redis
        client = redis.from_url(REDIS_URL, decode_responses=True,
                                socket_connect_timeout=2)
        client.ping()
        logger.info(f"Redis connecté : {REDIS_URL}")
        return client
    except Exception as e:
        logger.warning(f"Redis indisponible ({e}) — MagicMock activé")
        return MagicMock()   # silencieux si Redis absent

redis_client = _create_redis_client()
# ↑ Cet objet est mocké dans les tests via
mocker.patch('app.cache.redis_client')
```

4.4 Tester les services

```
Terminal — test manuel des services
# Tester pricing.py directement depuis le terminal Python
python -c "
from app.services.pricing import calcul_prix_ttc, appliquer_coupon,
calculer_total
from app.models import Coupon

print(calcul_prix_ttc(100.0))    # attendu : 120.0
print(calcul_prix_ttc(0.0))      # attendu : 0.0

coupon = Coupon(code='PROMO20', reduction=20.0, actif=True)
print(appliquer_coupon(120.0, coupon))   # attendu : 96.0

# calculer_total avec liste vide
print(calculer_total([]))   # attendu : 0.0
"
```

Relecture 4 : Services métier	**Commentaires**
4.1. Expliquez pourquoi les fonctions de pricing.py (calcul_prix_ttc, appliquer_coupon) ne prennent pas de session SQLAlchemy en paramètre, contrairement à reserver_stock(). Quelle est la règle de design derrière cette différence ?	Pas de session dans pricing.py parceque calcul_prix_ttc() et appliquer_coupon() sont de la logique pure, elles transforment des nombres sans toucher à la BDD. reserver_stock() modifie la BDD, elle a besoin d'une session. Règle : une fonction qui ne lit ni n'ecrit dans une BDD n'a pas besoin de session.
4.2. Dans cache.py, quand Redis n'est pas démarré, la fonction retourne MagicMock(). Expliquez ce qu'est un MagicMock et pourquoi c'est utile ici pour le développement sans Docker.	MagicMock ets un objet Python de la bibliothèque standard (`unittest.mock`) qui accepte n'importe quel appel de méthode sans lever d'exception. `redis_client.get('any_key')`, `redis_client.set('k','v')`, `redis_client.delete('k')` retournent tous `MagicMock()` silencieusement.

	Si Redis n'est pas démarré (développement sans Docker), l'import de `app.cache` n'échoue pas. L'API démarre normalement, le cache est simplement inopérant (miss systématique). Le développeur peut travailler sans avoir Redis installé. En test (TP1) : on remplace `redis_client` par un MagicMock contrôlé via `mocker.patch('app.cache.redis_client')`. On peut alors vérifier que `redis_client.delete()` est bien appelé sans avoir Redis.
4.3. Quel est l'avantage de séparer la logique métier (services/) des routes (routes/) du point de vue des tests ? Donnez un exemple concret avec reserver_stock().	Les services peuvent être testés sans démarrer FastAPI, sans HTTP, sans TestClient. On importe directement la fonction et on l'appelle avec des arguments Python. **Exemple avec reserver_stock() :** en test unitaire (TP1), on appelle `reserver_stock(product, 3, session)` directement avec un product SQLite en mémoire et un MagicMock Redis. Aucune route HTTP n'est impliquée. En test d'intégration (TP2), on passe par l'endpoint `POST /orders/` qui appelle reserver_stock() indirectement. Si reserver_stock() échoue en test unitaire, le bug est localisé dans le service. Si le test d'intégration échoue mais pas le test unitaire, le bug est dans la route ou le schéma, diagnostic plus rapide.

Partie 5 : API FastAPI et Vérification

Vous allez maintenant assembler toutes les pièces dans `app/main.py`, lancer l'API et explorer la documentation interactive générée automatiquement par FastAPI.

5.1 Remplir app/main.py

app/main.py (à copier intégralement)

```
# app/main.py
import logging
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.database import engine, Base
from app.routes import products, cart, orders, coupons

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s  %(levelname)-8s  %(message)s")
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Exécuté au démarrage et à l'arrêt de l'API."""
    logger.info("ShopFlow démarrage — création des tables...")
    Base.metadata.create_all(bind=engine)   # crée les tables SQLite
    logger.info("Tables créées. API prête.")
    yield   # l'API tourne ici
    logger.info("ShopFlow arrêt.")

app = FastAPI(
    title="ShopFlow API",
    description="API e-commerce pédagogique — Fil rouge Automatisation des
Tests",
    version="0.1.0",
    lifespan=lifespan,
)

app.add_middleware(CORSMiddleware, allow_origins=["*"],
    allow_methods=["*"], allow_headers=["*"])

app.include_router(products.router)   # /products
app.include_router(cart.router)       # /cart
app.include_router(orders.router)     # /orders
app.include_router(coupons.router)    # /coupons

@app.get("/health", tags=["health"])
def health_check():
    return {"status": "ok", "version": "0.1.0", "db": "sqlite"}

@app.get("/", tags=["health"])
def root():
    return {"message": "ShopFlow API — /docs pour la documentation"}
```

5.2 Lancer l'API et explorer Swagger

Ajouter les routes (voir fichiers pieces jointes) et lancer l'API

```
Terminal
# Lancer l'API en mode développement (hot-reload)
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Sortie attendue :
# INFO     ShopFlow démarrage — création des tables...
# INFO     Tables créées. API prête.
# INFO     Uvicorn running on http://0.0.0.0:8000
# INFO     Application startup complete.
```

URLs à explorer dans votre navigateur

- http://localhost:8000/health → doit retourner {"status": "ok", "version": "0.1.0", "db": "sqlite"}
- http://localhost:8000/docs → Swagger UI : documentation interactive (testez les endpoints ici)
- http://localhost:8000/redoc → ReDoc : documentation alternative plus lisible
- http://localhost:8000/ → message d'accueil de l'API

5.3 Tester l'API via Swagger ou curl

Depuis la Swagger UI (`/docs`) ou le terminal, effectuez les opérations suivantes dans l'ordre :

```
Scénario complet de test manuel
# Étape 1 : créer un produit
curl -X POST http://localhost:8000/products/ \
  -H 'Content-Type: application/json' \
  -d '{"name": "Laptop Pro", "price": 999.99, "stock": 10}'
# Réponse attendue : {"id": 1, "name": "Laptop Pro", ...}

# Étape 2 : lister les produits
curl http://localhost:8000/products/
# Réponse attendue : [{"id": 1, "name": "Laptop Pro", ...}]

# Étape 3 : ajouter au panier (user_id=42)
curl -X POST 'http://localhost:8000/cart/?user_id=42' \
  -H 'Content-Type: application/json' \
  -d '{"product_id": 1, "quantity": 2}'

# Étape 4 : voir le panier
curl http://localhost:8000/cart/42
# Réponse attendue : panier avec 1 article, sous_total calculé

# Étape 5 : passer la commande
curl -X POST http://localhost:8000/orders/ \
  -H 'Content-Type: application/json' \
  -d '{"user_id": 42}'
# Réponse attendue : commande avec status=pending, total_ttc calculé

# Étape 6 : vérifier que le panier est vidé
curl http://localhost:8000/cart/42
```

```
# Réponse attendue : panier vide (items: [])
```

5.4 Vérification finale

Terminal — vérification finale

```
# Vérifier que le projet est complet et qu'aucun import ne manque
python -c "
from app.main import app
from app.database import engine, Base, get_db
from app.models import Product, Cart, CartItem, Order, OrderItem, Coupon
from app.schemas import ProductCreate, ProductResponse
from app.schemas import CartResponse, OrderCreate, OrderResponse
from app.services.pricing import calcul_prix_ttc, appliquer_coupon,
calculer_total
from app.services.stock import verifier_stock, reserver_stock, liberer_stock
print('Tous les imports OK — ShopFlow est prêt')
"
```